

```
public void metodo() {
```

Curso de Java

Aula 4

Métodos em Java

Criando, chamando e organizando seu código

```
System.out.println("Bora codar! ☕");
```

```
if (condição) {
```

```
return resultado;
```



Criação
De Métodos



Parâmetros
E Retorno



Modularização
De Código



Debugging
No NetBeans

<> O que são Métodos?

Blocos de Código



Reutilizáveis

Agrupam instruções que executam uma tarefa específica



Reutilização

Evite repetir código



Organização

Divida problemas em partes



Manutenção

Corrija erros em um lugar

Exemplo Simples

```
// Define o método  
  
public static void saudar() {  
    System.out.println("Olá, Java!");  
}  
  
// Usa o método no main  
  
public static void main(String[] args)  
{  
    saudar(); // Executa o código  
}
```



Analogia: Receita de Cozinha

Um método é como uma receita: você define os passos uma vez e pode usá-la sempre que precisar!

+ Criação de Métodos

Estrutura Básica

```
public static int somar(int a, int b)
{
    return a + b;
}
```

1 Modificador: `public static`

2 Retorno: `int`

3 Nome: `somar`

4 Parâmetros: (`a`, `b`)

Exemplos Práticos

Sem Retorno (void)

```
public static void exibirMensagem() {
    System.out.println("Aula 4!");
}
```

Com Retorno

```
public static double calcularMedia(double
n1, double n2) {
    return (n1 + n2) / 2;
}
```

 **void:** não retorna valor • **tipo:** retorna aquele tipo

Escopo de Variáveis

Onde a variável pode ser acessada

```
public static void exemplo() {  
    int x = 10; // escopo local  
    if (x > 5) {  
        String msg = "Grande";  
        // msg visível aqui  
    }  
  
    // msg NÃO visível aqui ✕  
    System.out.println(x);  
}
```

- 1 Local: criada dentro do método
- 2 Bloco: visível apenas naquele bloco

Sobre carga de Métodos

Mesmo nome, parâmetros diferentes

```
// Versão 1: dois inteiros public static  
int somar(int a, int b) {  
    return a + b;  
}  
  
// Versão 2: três inteiros public static  
int somar(int a, int b, int c) {  
    return a + b + c;  
}  
  
// Versão 3: doubles public static double  
somar(double a, double b) {  
    return a + b;  
}
```

Chamando:

somar(2, 3) → 5

somar(2, 3, 4) → 9

somar(2.5, 3.5) → 6.0

Modularização de Código

❌ Código Monolítico ❌

Tudo em um método só

```
public static void processarTudo() {  
    // Ler dados Scanner sc = new  
    Scanner(System.in);  
    System.out.println("Nome:"); String nome =  
    sc.nextLine();  
  
    // Calcular double nota1 = sc.nextDouble();  
    double nota2 = sc.nextDouble(); double media  
    = (nota1 + nota2) / 2;  
  
    // Exibir System.out.println(nome + ": " +  
    media); if (media >= 7) {  
        System.out.println("Aprovado!");  
    }  
}
```

⚠️ Problemas: difícil ler, testar e manter

✅ Código Modularizado ✅

Dividido em métodos menores

```
static String lerNome() {  
    // lógica para ler  
}
```

```
static double calcularMedia(double n1, double n2) {  
    // lógica para calcular  
}
```

```
static void exibirResultado(String nome, double  
media) {  
    // lógica para exibir  
}
```



Fácil de ler



Fácil de testar

Prática: Refatoração

✗ Antes: Código Repetitivo

```
public static void main(String[] args) {  
    // Cálculo média João double n1Joao = 8.5;  
    double n2Joao = 7.0; double mediaJoao =  
    (n1Joao + n2Joao) / 2;  
    System.out.println("João: " + mediaJoao);  
  
    // Cálculo média Maria double n1Maria = 9.0;  
    double n2Maria = 8.5; double mediaMaria =  
    (n1Maria + n2Maria) / 2;  
    System.out.println("Maria: " + mediaMaria);  
  
    // Cálculo média Pedro double n1Pedro = 6.0;  
    double n2Pedro = 7.5; double mediaPedro =  
    (n1Pedro + n2Pedro) / 2;  
    System.out.println("Pedro: " + mediaPedro);  
}
```

⚠ Mesma lógica repetida 3 vezes!

✓ Depois: Método Reutilizável

```
static void exibirMedia(String nome, double n1,  
double n2) {  
    double media = (n1 + n2) / 2;  
    System.out.println(nome + ": " + media);  
}
```

```
public static void main(String[] args) {  
    exibirMedia("João", 8.5, 7.0);  
    exibirMedia("Maria", 9.0, 8.5);  
    exibirMedia("Pedro", 6.0, 7.5);  
}
```

Benefícios da Refatoração:

- ✓ Código mais limpo
- ✓ Fácil manutenção
- ✓ Menos duplicação
- ✓ Reutilização

Debugging no NetBeans

Como Debugar Métodos

↓ Step Into (F7)

Entra dentro do método

Usado para ver o código interno do método linha por linha

→ Step Over (F8)

Pula o método inteiro

Usado quando não precisa ver o código interno, só o resultado



Dica: Use breakpoint (clique na margem) para pausar a execução

<> Exemplo Prático

```
public static void main(String[] args) {  
    ● int resultado = calcular(5, 3); ← breakpoint  
    System.out.println(resultado);  
}  
  
public static int calcular(int a, int b) {  
    int soma = a + b; int dobro = soma * 2;  
    return dobro;  
}
```

F7 – Step Into

Vê cada linha de
`calcular()`

F8 – Step Over

Pula direto para resultado
= 16

Atalhos no NetBeans:

Ctrl+F5
Debug

F5
Continue

F7 Step
Into

F8 Step
Over

Coesão e Acoplamento

✓ Coesão

Um método, uma responsabilidade

Alta Coesão ✓

```
static void calcularMedia(double n1, double n2) {  
    double media = (n1 + n2) / 2; return media;  
}
```

Baixa Coesão X

```
static void processarTudo() {  
    // ler, calcular, exibir, salvar... //  
    Muitas responsabilidades!  
}
```

Acoplamento

Menos dependência entre métodos

✓ Baixo Acoplamento ✓



Independentes • Fáceis de manter

✗ Alto Acoplamento X



Muito dependentes • Difícil manter

Exercício 1 – Tarefa Jira

 JAVA-101

FÁCIL

Calculadora de Média Escolar

TO DO

Backend

História do Usuário

Como professor, preciso calcular a média de três notas dos alunos para determinar seu desempenho acadêmico.

Contexto

A escola TechEdu está digitalizando seu sistema de notas. Atualmente, os professores calculam manualmente as médias, o que é lento e propenso a erros.

Detalhes da Tarefa

Descrição

Crie um método `calcularMedia` que receba três notas como parâmetros e retorne a média aritmética.

Critérios de Aceite

- ✓ O método deve receber três notas como parâmetros
- ✓ Deve retornar a média aritmética
- ✓ O método deve ser chamado no main e exibir o resultado

 Dica: Use o tipo double para permitir notas decimais

Exercício 2 – Tarefa Jira

 **JAVA-102**

FÁCIL

Calculadora de Áreas

TO DO

Backend

História do Usuário

Como arquiteto, preciso calcular a área de diferentes formas geométricas para estimar os materiais necessários na construção.

Contexto

A construtora BuildTech está desenvolvendo um sistema para calcular áreas de terrenos e cômodos. Atualmente, cada tipo de forma geométrica possui um cálculo manual diferente, causando confusão e erros.

Detalhes da Tarefa

Descrição

Crie métodos sobrecarregados chamados `calcularArea` para:



Quadrado
1 parâmetro



Retângulo
2 parâmetros



Círculo
1 parâmetro

Critérios de Aceite

- ✓ Todos os métodos devem ter o mesmo nome: `calcularArea`
- ✓ Cada método deve ter parâmetros diferentes (sobrecarga)
- ✓ Deve demonstrar o uso de cada método no main

 Dica: Sobrecarga = mesmo nome, parâmetros diferentes

Exercício 3 – Tarefa Jira

JAVA-103

FÁCIL

Refatoração de Código Legado

TO DO

Backend

História do Usuário

Como desenvolvedor sênior, preciso refatorar o código legado do sistema de gestão escolar para melhorar sua manutenibilidade e legibilidade.

Contexto

A EscolaTech herdou um sistema antigo onde todo o código está no método main. O cliente solicitou uma refatoração urgente, pois a equipe atual está tendo dificuldades para entender e manter o código.

<> Código para Refatorar

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Cadastrar aluno System.out.println("Nome do
aluno:"); String nome = sc.nextLine();
System.out.println("Matrícula:"); String
matricula = sc.nextLine();

    // Cadastrar notas System.out.println("Quantas
notas?"); int qtd = sc.nextInt(); double[] notas
= new double[qtd];

    for (int i = 0; i < qtd; i++) {
        System.out.println("Nota " + (i+1) + ":");
        notas[i] = sc.nextDouble();
    }

    // Calcular média double soma = 0; for (double
nota : notas) {
        soma += nota;
    } double media = soma / qtd;

    // Verificar situação String situacao; if (media
>= 7.0) {
        situacao = "Aprovado";
    } else if (media >= 5.0) {
        situacao = "Recuperação";
    } else {
        situacao = "Reprovado";
    }

    // Exibir boletim System.out.println("=== BOLETIM
==="); System.out.println("Aluno: " + nome);
System.out.println("Matrícula: " + matricula);
System.out.println("Média: " + media);
System.out.println("Situação: " + situacao);
}
```



Instruções para Refatoração:

Crie métodos separados para: cadastrar aluno, cadastrar notas, calcular média, verificar situação e exibir boletim.

Exercício 4 – Tarefa Jira (Parte 1)

 **JAVA-104**

MÉDIO

Sistema de Estoque com CRUD

TO DO

Backend

História do Usuário

Como gerente de estoque, preciso de um sistema completo para gerenciar produtos e categorias, podendo cadastrar, consultar, alterar e excluir itens.

Contexto

A MegaStore precisa de um sistema robusto com operações CRUD (Create, Read, Update, Delete) para manter o estoque sempre atualizado.

Conceito CRUD:

C (Create) | R (Read) | U (Update) | D (Delete)

Menu Principal

1 **Gerenciar Produtos** **CRUD**
Sub-menu com operações CRUD

2 **Gerenciar Categorias** **CRUD**
Sub-menu com operações CRUD

3 **Relatórios**
Relatórios de estoque e categorias

0 **Sair**
Encerra o sistema

 **Atenção:**
As opções 1 e 2 possuem sub-menus com operações CRUD. Veja detalhes no próximo slide.

📋 Exercício 4 – Tarefa Jira (Parte 2)

📁 Sub-Menu: Produtos

↳ Opção 1 - Gerenciar Produtos

1. Inserir Produto CREATE

Nome: Notebook | Preço: 3500.00
Quantidade: 15 | Categoria: Eletrônicos
✓ Produto cadastrado com sucesso!

2. Listar Produtos READ

```
=== LISTA DE PRODUTOS ===  
1. Notebook | R$3500.00 | Qtde: 15 | Eletrônicos  
2. Mouse | R$80.00 | Qtde: 5 | Periféricos
```

⚠️ Produtos com estoque baixo: Mouse (5 unidades)

3. Alterar Produto UPDATE

Número do produto: 1
Novo preço: 3200.00
✓ Produto atualizado com sucesso!

4. Excluir Produto DELETE

Número do produto: 2
Confirmar exclusão (S/N)? S
✓ Produto excluído com sucesso!

0. Voltar BACK

Retornando ao menu principal...

📁 Sub-Menu: Categorias

↳ Opção 2 - Gerenciar Categorias

1. Inserir Categoria CREATE

Nome: Eletrônicos
✓ Categoria cadastrada com sucesso!

2. Listar Categorias READ

```
=== LISTA DE CATEGORIAS ===  
1. Eletrônicos (2 produtos)  
2. Periféricos (1 produto)
```

3. Alterar Categoria UPDATE

Número da categoria: 2
Novo nome: Acessórios
✓ Categoria atualizada com sucesso!

4. Excluir Categoria DELETE

Número da categoria: 3
Confirmar exclusão (S/N)? S
✓ Categoria excluída com sucesso!

0. Voltar BACK

Retornando ao menu principal...



Importante:

Não permitir excluir categoria com produtos vinculados. Exibir mensagem de erro.

Exercício 4 – Tarefa Jira (Parte 3)

Opção 3: Relatórios

1 Relatório de Estoque

```
=== RELATÓRIO DE ESTOQUE ===  
Total de produtos: 65  
Valor total em estoque: R$57.500,00  
Produtos com estoque baixo: 2
```

```
Valor por categoria:  
- Eletrônicos: R$52.500,00 (3 itens)  
- Periféricos: R$5.000,00 (2 itens)
```

2 Relatório por Categoria

```
=== RELATÓRIO POR CATEGORIA ===  
Categoria: Eletrônicos  
- Notebook | Qtde: 15 | Valor: R$52.500,00  
- Smartphone | Qtde: 20 | Valor: R$40.000,00
```

```
Total na categoria: 35 itens - R$92.500,00
```

0 Voltar ao Menu Principal

```
Retornando ao menu principal...
```

Critérios de Aceite

- ✓ Menu principal deve funcionar em loop até escolher "Sair"
- ✓ Sub-menus também devem funcionar em loop com opção "Voltar"
- ✓ Usar Scanner para ler todas as entradas do usuário
- ✓ Implementar operações CRUD completas para produtos e categorias
- ✓ Criar métodos separados para cada funcionalidade
- ✓ Relatórios devem calcular valor total do estoque por categoria
- ✓ Não permitir excluir categoria com produtos vinculados

 **Desafio Extra:**
Implementar validação de dados (ex: preço não pode ser negativo, quantidade mínima de estoque)